

These weights correspond to a **pre-norm Transformer block** using **Grouped Query Attention (GQA)** and a **SwiGLU feed-forward network**, which is what Qwen3-VL uses.

Let's go through the forward pass mathematically.

1. Input

Suppose the input is

$$X \in \mathbb{R}^{L \times 2048}$$

where

- L = sequence length
- 2048 = hidden size

2. Attention RMSNorm

```
blk.0.attn_norm.weight [2048]
```

This is an RMSNorm.

$$X_n = \text{RMSNorm}(X; \gamma_{attn})$$

where

$$\gamma_{attn} \in \mathbb{R}^{2048}$$

3. Linear projections

Query

`attn_q.weight [2048,2048]`

$$Q = X_n W_Q$$

with

$$W_Q \in \mathbb{R}^{2048 \times 2048}$$

so

$$Q \in \mathbb{R}^{L \times 2048}$$

Key

`attn_k.weight [2048,1024]`

$$K = X_n W_K$$

where

$$W_K \in \mathbb{R}^{2048 \times 1024}$$

giving

$$K \in \mathbb{R}^{L \times 1024}$$

Value

`attn_v.weight [2048,1024]`

$$V = X_n W_V$$

also

$$V \in \mathbb{R}^{L \times 1024}$$

4. Per-head normalization

Qwen applies RMSNorm **after** projection.

attn_q_norm.weight [128]

attn_k_norm.weight [128]

Why only 128?

Because

$$2048 = 16 \times 128$$

so there are **16 query heads**.

For keys,

$$1024 = 8 \times 128$$

so there are **8 KV heads**.

Each head has

$$d_{head} = 128.$$

For every query head

$$Q_i = \text{RMSNorm}(Q_i; \gamma_q)$$

For every key head

$$K_j = \text{RMSNorm}(K_j; \gamma_k)$$

The same 128-element normalization weight is shared across heads.

5. Reshape into heads

Queries

$$Q \rightarrow (L, 16, 128)$$

Keys

$$K \rightarrow (L, 8, 128)$$

Values

$$V \rightarrow (L, 8, 128)$$

6. Grouped Query Attention

There are

- 16 query heads

- 8 KV heads

Thus every KV head serves 2 query heads.

Conceptually,

```
Q0,Q1 -> KV0
Q2,Q3 -> KV1
...
Q14,Q15 -> KV7
```

For each query head

$$A_i = \text{softmax} \left(\frac{Q_i K_{g(i)}^T}{128} \right)$$

where

$g(i)$ maps query heads to KV heads.

Output

$$O_i = A_i V_{g(i)}$$

7. Concatenate heads

After attention

$$O = \text{Concat}(O_1, \dots, O_{16})$$

so

$$O \in \mathbb{R}^{L \times 2048}$$

8. Output projection

```
attn_output.weight [2048,2048]
```

$$Y = OW_O$$

where

$$W_O \in \mathbb{R}^{2048 \times 2048}$$

9. Residual

$$X_1 = X + Y$$

10. FFN RMSNorm

`ffn_norm.weight [2048]`

$$X_f = \text{RMSNorm}(X_1)$$

11. SwiGLU feed-forward

The FFN consists of three matrices.

Up projection

`ffn_up.weight [2048,6144]`

$$U = X_f W_{up}$$

$$U \in \mathbb{R}^{L \times 6144}$$

Gate projection

`ffn_gate.weight [2048,6144]`

$$G = X_f W_{gate}$$

SwiGLU activation

$$H = \text{SiLU}(G) \odot U$$

where

$$\text{SiLU}(x) = x\sigma(x)$$

and $\odot$ denotes elementwise multiplication.

Down projection

`ffn_down.weight [6144,2048]`

$$F = HW_{down}$$

returning

$$F \in \mathbb{R}^{L \times 2048}$$

12. Final residual

$$X_{out} = X_1 + F$$

This becomes the input to the next transformer block.

Complete block equation

Putting everything together:

$$\begin{aligned} X_n &= \text{RMSNorm}(X) \\ Q &= X_n W_Q \\ K &= X_n W_K \\ V &= X_n W_V \\ Q &\leftarrow \text{HeadRMSNorm}(Q) \\ K &\leftarrow \text{HeadRMSNorm}(K) \\ O &= \text{GQA}(Q, K, V) \\ X_1 &= X + OW_O \\ X_f &= \text{RMSNorm}(X_1) \\ U &= X_f W_{up} \\ G &= X_f W_{gate} \\ H &= \text{SiLU}(G) \odot U \\ F &= HW_{down} \\ X_{out} &= X_1 + F \end{aligned}$$

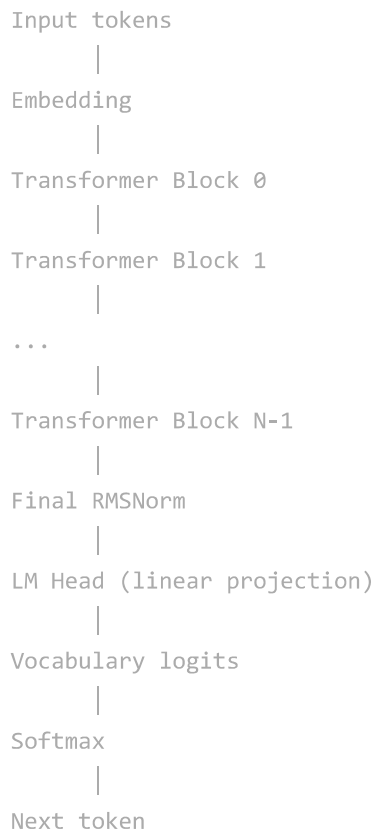
This sequence directly matches the listed tensors:

- `attn_norm` → pre-attention RMSNorm
- `attn_q` , `attn_k` , `attn_v` → Q/K/V projections
- `attn_q_norm` , `attn_k_norm` → per-head RMSNorm for Q and K
- `attn_output` → output projection after GQA
- `ffn_norm` → pre-FFN RMSNorm
- `ffn_up` , `ffn_gate` , `ffn_down` → SwiGLU feed-forward network.

what does happen at last layer? How a token is predicted?

The transformer blocks produce increasingly refined hidden representations. **The last layer does not directly output a token.** Instead, it outputs a vector that is converted into a probability distribution over the vocabulary.

For Qwen3-VL-2B, the process is approximately:



1. Last transformer block

Suppose after the final block you have

$$H \in \mathbb{R}^{L \times 2048}$$

Each row is the hidden representation for one token.

Example:

Token:	The	cat	sat
Vector:	h_0	h_1	h_2

Each h_i is a 2048-dimensional vector.

2. Final normalization

Most modern LLMs (including Qwen, Llama, Mistral) apply one more RMSNorm.

$$\hat{H} = \text{RMSNorm}(H)$$

3. LM Head

There is one final matrix

$$W_{lm}$$

whose shape is

$$2048 \times V$$

where

- V = vocabulary size (around 1.5×10^5 for Qwen3).

The logits are

$$Z = \hat{H}W_{lm}$$

For a single token

$$z = hW_{lm}$$

which produces one score per vocabulary token.

If

$$V = 151936,$$

then

$$z \in \mathbb{R}^{151936}.$$

Every entry corresponds to one token in the tokenizer.

For example,

"the"	7.5
"cat"	5.3
"dog"	8.1
". "	2.8
...	

These are **logits**, not probabilities.

4. Softmax

Convert logits into probabilities:

$$P(i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Now every vocabulary token has a probability.

Example:

Token	Probability
dog	0.55
cat	0.20
mouse	0.05
.	0.01
...	...

5. Choose the next token

Several decoding methods are possible:

Greedy

Choose

$$\arg \max_i P(i)$$

Always pick the most probable token.

Sampling

Randomly sample according to the probabilities.

Top-k

Keep only the k highest-probability tokens.

Top-p (nucleus)

Keep the smallest set of tokens whose cumulative probability exceeds p .

The selected token is appended to the sequence.

6. Feed it back

Suppose the model predicts

dog

The sequence becomes

The cat saw a dog

This entire sequence is then fed through the transformer again (or, during efficient generation, only the new token is processed while cached keys and values from previous tokens are reused). The model predicts the next token after "dog" .

Why only the last token matters during generation

Suppose the sequence length is 10:

t_0 t_1 t_2 t_3 t_4 t_5 t_6 t_7 t_8 t_9

The transformer produces

h_0 h_1 h_2 h_3 h_4 h_5 h_6 h_7 h_8 h_9

The LM head computes logits for **every position**:

z_0 z_1 z_2 z_3 z_4 z_5 z_6 z_7 z_8 z_9

During autoregressive generation, only z_9 is used to predict the next token t_{10} . The logits for earlier positions are ignored because their "next token" targets were already known.

During training, however, logits from **every position** are used. Each position predicts the token immediately following it:

Hidden state	Predicts
h_0	token 1
h_1	token 2

Hidden state	Predicts
h_2	token 3
...	...
h_{L-2}	last token
h_{L-1}	token after the sequence (or end-of-sequence)

So, mathematically, the next-token prediction for the current context is simply:

$$\text{next token} = \text{Decode}(\text{softmax}(\text{RMSNorm}(h_{\text{last}}) W_{lm}))$$

where h_{last} is the 2048-dimensional hidden state of the final input token, W_{lm} maps that hidden state to vocabulary logits, and `Decode` represents the chosen decoding strategy (greedy, top-k, top-p, sampling, etc.).



what is `qwen3vl.vision.spatial_merge_size = 2`?

`qwen3vl.vision.spatial_merge_size = 2` is a **vision encoder parameter**. It tells the model to **merge neighboring image patches in 2×2 spatial groups** before sending them through most of the Vision Transformer (ViT). This reduces the number of visual tokens and computation.

Let's see why.

Without spatial merging

Suppose the image is split into patches of size 14×14.

For a 448×448 image,

$$448/14 = 32$$

so the patch grid is

$$32 \times 32 = 1024$$

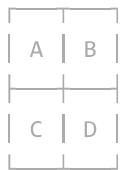
visual tokens.

□□□□□□□□□□□□□□□□□□□□□□□□
 □□□□□□□□□□□□□□□□□□□□□□□□
 □□□□□□□□□□□□□□□□□□□□□□□□
 ...
 32 × 32 patches

The transformer would process **1024 visual tokens**.

With `spatial_merge_size = 2`

Now every **2×2 block** of neighboring patches is merged.



↓

One merged token

So

32 × 32 patches

↓

16 × 16 merged tokens

The number of tokens becomes

$$16 \times 16 = 256.$$

So the visual sequence is reduced by 4×.

Mathematical view

Suppose the patch embeddings are

$$x_{ij} \in \mathbb{R}^D.$$

A 2×2 neighborhood is

$$x_{00}, x_{01}, x_{10}, x_{11}.$$

These are combined into one feature vector. A common implementation is:

1. Concatenate the four embeddings:

$$z = [x_{00}; x_{01}; x_{10}; x_{11}] \in \mathbb{R}^{4D}$$

2. Apply a learned linear projection:

$$y = zW$$

where

$$W \in \mathbb{R}^{4D \times D}.$$

The output y is a single embedding of dimension D .

(Architecturally, Qwen3-VL uses a learned merge module rather than simply averaging.)

Why do this?

Transformer attention has quadratic complexity:

$$O(N^2)$$

where N is the number of tokens.

Reducing

$$1024 \rightarrow 256$$

reduces attention cost by

$$\left(\frac{256}{1024} \right)^2 = \frac{1}{16}.$$

So a 4× reduction in tokens yields about a **16× reduction in attention computation** for the layers operating on the merged sequence.

Why not merge everything?

If you merged too aggressively (e.g., 4×4), you'd lose fine spatial details like small text or thin object boundaries.

A merge size of 2 is a compromise:

- preserves most local information,
- greatly reduces computation,
- keeps enough resolution for tasks like OCR and object localization.

Summary

`qwen3vl.vision.spatial_merge_size = 2` means the vision encoder **groups every 2×2 neighboring patch embeddings into a single visual token** before further processing. If the original patch grid is $H \times W$, the merged grid becomes

$$\frac{H}{2} \times \frac{W}{2},$$

reducing the number of visual tokens by a factor of $2^2 = 4$, which significantly lowers the computational cost of the subsequent transformer layers.



What is the type of vision transformer used in Qwen2-vl?
Give all mathematical formulas.

Qwen2-VL uses a **Vision Transformer (ViT)** as its visual backbone (about 600M parameters for all model sizes), but it is **not** a vanilla ViT. It introduces three important architectural changes:

1. **Naive Dynamic Resolution (NDR)** – arbitrary image/video resolutions produce a variable number of visual tokens.
2. **Multimodal Rotary Position Embedding (M-RoPE)** – rotary embeddings generalized to temporal, height, and width dimensions.
3. **Patch Merger** – after the vision transformer, neighboring spatial tokens are merged before being sent to the language model. [🔗 arXiv +2](#)

Below is the mathematical pipeline.

1. Image

Let

$$I \in \mathbb{R}^{H \times W \times 3}$$

For videos,

$$V \in \mathbb{R}^{T \times H \times W \times 3}.$$

2. Dynamic resizing

Unlike CLIP, Qwen2-VL does **not** resize every image to a fixed resolution (e.g., 224×224).

Instead, it rescales the image so that

$N_{\text{pixels}} \in [\text{min_pixels}, \text{max_pixels}]$

while approximately preserving aspect ratio.

Therefore

$$(H, W) \rightarrow (H', W').$$

The number of visual tokens varies from image to image. [🔗 arXiv +1](#)

3. Patch embedding

Patch size

$$P = 14.$$

Video temporal patch size

$$P_t = 2.$$

For images,

$$N_h = \frac{H'}{14}$$
$$N_w = \frac{W'}{14}$$

Number of patches

$$N = N_h N_w.$$

Each patch is flattened

$$x_i \in \mathbb{R}^{14 \times 14 \times 3}$$

then projected

$$e_i = x_i W_E + b$$

where

$$W_E \in \mathbb{R}^{588 \times D}$$

because

$$14 \times 14 \times 3 = 588.$$

The embedding dimension is

$$D = 1280$$

for the released models.

Thus

$$E = [e_1, \dots, e_N] \in \mathbb{R}^{N \times 1280}.$$

4. Vision Rotary Position Embedding

Instead of adding learned position embeddings,

Qwen2-VL uses rotary embeddings.

For a 2D image,

each token has coordinates

$$(h, w).$$

For videos,

$$(t, h, w).$$

The rotary angle is computed independently for each axis.

For one frequency

$$\theta_i = 10000^{-2i/d}$$

then

$$\phi_h = h\theta_i$$

$$\phi_w = w\theta_i$$

(and similarly for t in videos).

The embedding rotates every pair of features.

Given

$$q = (q_1, q_2),$$

RoPE becomes

$$\begin{aligned} q'_1 &= q_1 \cos \phi - q_2 \sin \phi \\ q'_2 &= q_1 \sin \phi + q_2 \cos \phi. \end{aligned}$$

The same rotation is applied to keys.

This allows attention to encode relative spatial relationships naturally.  [arXiv +1](#)

5. Transformer block

Input

$$X \in \mathbb{R}^{N \times 1280}.$$

Each vision block performs

RMSNorm

$$X_n = \text{RMSNorm}(X).$$

Query

$$Q = X_n W_Q.$$

Key

$$K = X_n W_K.$$

Value

$$V = X_n W_V.$$

Apply RoPE

$$Q \leftarrow \text{RoPE}(Q)$$

$$K \leftarrow \text{RoPE}(K).$$

Attention

For every head

$$A = \text{softmax} \begin{pmatrix} QK^T \\ d \end{pmatrix}.$$

Output

$$O = AV.$$

Output projection

$$Y = OW_O.$$

Residual

$$X_1 = X + Y.$$

Feed-forward

Normalize

$$X_f = \text{RMSNorm}(X_1).$$

MLP

$$F = W_2 \text{ GELU}(W_1 X_f).$$

Residual

$$X_{out} = X_1 + F.$$

Repeat for all vision layers.  [GitHub](#)

6. Spatial Patch Merger

After the transformer,

neighboring

$$2 \times 2$$

patches are merged.

Suppose four neighboring embeddings are

$$x_1, x_2, x_3, x_4.$$

Concatenate

$$z = [x_1; x_2; x_3; x_4]$$

so

$$z \in \mathbb{R}^{4D}.$$

Then apply a learned projection

$$y = zW_M + b.$$

Thus

4 tokens $\rightarrow$ 1 token.

If the original grid is

$$N_h \times N_w,$$

the merged grid becomes

$$\frac{N_h}{2} \times \frac{N_w}{2}.$$

Hence the number of visual tokens decreases by a factor of four before entering the language model. This corresponds to the configuration parameter `spatial_merge_size =`

2 .  GitHub +1

7. Projection to the LLM

After merging,

the visual embeddings are projected into the language model's hidden dimension.

If

$$D_v = 1280,$$

and the language model uses

$$D_l = 1536$$

(for the 2B model),

then

$$X_{LLM} = X_{vision} W_P.$$

Now every visual token has exactly the same dimensionality as a text token.

8. Mixed sequence

Suppose the prompt is

```
<image>  
Describe the object.
```

The language model receives

[V1][V2][V3]...[Vk]

Describe

the

object

.

where

$$V_i$$

are the projected visual tokens.

The decoder transformer then performs standard causal self-attention over this combined sequence, allowing text tokens to attend to the preceding visual tokens.

Overall computation

The vision encoder can be summarized as

$I \rightarrow$ Dynamic Resize
 $\rightarrow$ Patch Embed
 $\rightarrow$ M-RoPE
 $\rightarrow L \times$ ViT Blocks
 $\rightarrow 2 \times 2$ Patch Merger
 $\rightarrow$ Linear Projection
 $\rightarrow$ LLM Visual Tokens

Compared with a standard ViT, the main innovations in Qwen2-VL are the **dynamic-resolution tokenization**, **multimodal rotary positional encoding over temporal and spatial dimensions**, and the **learned 2×2 spatial patch merger**, which together let the model process arbitrary-resolution images and videos efficiently before passing visual tokens into the decoder LLM. [arXiv](#) +2



Sources